

Atlas Camera — Architecture Review

2026-07-23 · scope weighted to recent hot spots: `nodes_geometry.py`, `nodes_viewport.py`, `node_helpers.py`, `mesh_repair.py`, `viewport_payload.py`
Vocabulary: **module** / **interface** / **depth** (small interface hiding big implementation) / **seam** / **adapter** / **leverage** (one change, many beneficiaries) / **locality** (bug lives where the logic lives). Deletion test: would deleting the thing *concentrate* complexity, or just move it?

1 · Fold AtlasDeriveProjectionGeometry onto the shared relief-derivation seam

Strong

`nodes_geometry.py:238-413` (proj derive) `nodes_geometry.py:872-978` (relief derive) `node_helpers.py:436` (`_replace_proxy_role_geometry`)
`core/depth_geometry.py:533-578` (helper trio)

Problem. The derive-node family is deep and consistent through four shared helpers (`_solve_camera_params` / `_horizon_y_from_solve` / `_depth_map_for_solve` / `_replace_proxy_role_geometry`) — *except* the oldest node, `AtlasDeriveProjectionGeometry`, which bypasses all four with inline copies (lines 272-304, 400-411). Its relief branch duplicates `AtlasDeriveReliefMesh`'s build-call + stats + live-fill + hole-mask tail (~40 parallel lines). The cost is not hypothetical: **this week's sawtooth-fill feature shipped on the relief node and AtlasInput but is missing from the proj node** (its 4th widget never landed; a dead import was left behind), and the proj node calls the *uncached* `estimate_ground_scale` while every other path goes through the memoized `_ground_scale_cached`. A new relief widget today costs ~6 edits (3× `INPUT_TYPES`, 3× signature, 3× body).

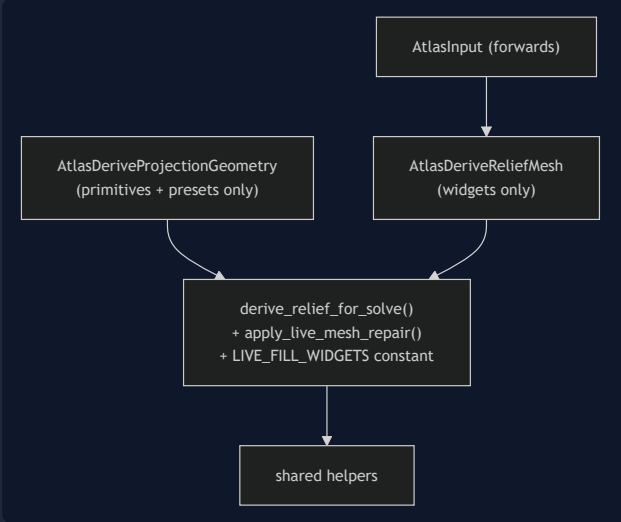
Solution. One shared relief-derivation helper (solve + depth-map + widget bundle in → mesh, stats, hole-mask out) that both nodes call, plus one shared "live mesh repair" step (interior fill + sawteeth + hole-mask recompute + stats) and one module-level live-fill widget-cluster constant reused by all three `INPUT_TYPES`. The proj node keeps its extra concern (primitives + scene_type presets) but stops re-implementing camera-params/horizon/depth/geometry-replacement.

Benefits. Locality: a live-fill bug fixes in one function, not three. Leverage: the next relief widget is 2 edits, not 6, and cannot drift per-node. Tests: the shared helper is testable once through its own interface instead of via three node harnesses; the existing per-node tests shrink to wiring assertions. Deletion test passes — deleting the inline copies concentrates complexity into helpers that already exist.

BEFORE — PROJ NODE FORKS EVERY SHARED CONCERN



AFTER — ONE DEEP SEAM, TWO THIN ADAPTERS



2 · Retire the hand-maintained copy inside derive_projection_proxies

Strong

`core/proxy_geometry.py:506-740` `core/depth_geometry.py:49-524`

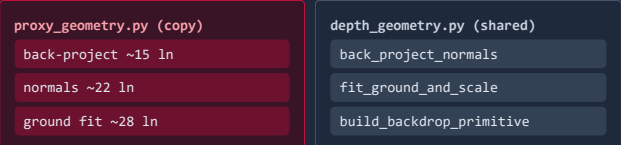
⚠ Contradicts a documented decision — `CLAUDE.md` records the copies were "left untouched to avoid regression risk" when `depth_geometry.py` was factored out. Worth reopening because the fork is now the *default* code path: `azimuth_walls` (the most-used extractor) runs the copy, while all newer extractors run the shared version — the drift risk the doc warned about is live, permanent, and growing.

Problem. ~115-120 lines of `derive_projection_proxies` are a by-hand copy of `back_project_normals` (:540-554), `normals+validity` (:556-577), `fit_ground_and_scale` (:582-609) and `build_backdrop_primitive` (:686-737). The same file also duplicates its own ground-primitive block twice (:618-640 vs :823-844). Every extractor must agree bit-for-bit on world points and metric scale for merges to align — today that agreement is maintained by hand.

Solution. Point `derive_projection_proxies` at the shared functions, guarded by a numerical-parity test first (same fixture through both paths, assert bit-identical points/scale/backdrop) so the swap is proven safe before any deletion. Fold the intra-file ground-primitive duplicate into one helper.

Benefits. Leverage: the "all extractors agree on the metric world" invariant becomes enforced by construction instead of by discipline. Locality: back-projection bugs are debugged in one module. The parity test survives as a regression pin. Deletion test passes cleanly — 140 lines disappear and nothing is relocated.

BEFORE — TWO SOURCES OF TRUTH



AFTER — ONE DEEP MODULE, FOUR CALLERS



backdrop ~52 ln
← runs the DEFAULT path

← used by 3 newer extractors

azimuth_walls (default)

vertical_extrusion

ransac_planes

room_cuboid

depth_geometry.py
back-project · ground fit ·
backdrop
(+ parity regression test)

3 · Decompose the viewport wire-protocol serializer

Worth exploring

comfy/viewport_payload.py:96-332

Problem. `_extract_blockout_camera` is a single 237-line flat function interleaving eight concerns: camera intrinsics + shot-cam render keys, orbit pivot, source-image b64, proxy geometry, projection-sources loop (with inlined band metrics + hidden-geometry provenance), vanishing points/horizon, HUD meta, and the 48-line 24-bit depth pack. It is the serialization boundary the frontend depends on, and every new payload key lands in the same scope — low locality, and untestable except end-to-end.

Solution. Keep the function as the single assembly point but extract the two heaviest self-contained blocks behind their own interfaces: `_serialize_projection_sources(solve)` and `_pack_primary_depth(depth)` (the depth pack already has its own try/except boundary). Payload keys stay byte-identical.

Benefits. Each extracted serializer is testable against a fixture solve without the full payload; the load-bearing conventions (fx/fy = PHOTO vs render_fy = VIEWING camera; NEAREST-sampled depth pack) get named homes instead of comment islands. Modest leverage — this is decomposition, not deletion, hence not "Strong".

BEFORE — ONE 237-LINE SCOPE

```
camera + shot-cam keys
pivot · source b64
proxies
projection_sources loop (~40 ln, 3 sub-concerns)
VPs + horizon
HUD meta
depth pack (~48 ln, own try/except)
flat dict assembly
```

AFTER — ASSEMBLY + TWO NAMED SERIALIZERS

```
_extract_blockout_camera (assembly, ~140 ln)
  _serialize_projection_sources() — unit-testable
  _pack_primary_depth() — unit-testable
payload keys byte-identical
```

4 · One segmentation-cascade policy

Worth exploring

nodes_viewport.py:754-769

nodes_inpaint.py:667-684

node_helpers.py:693 (`_native_sam3_available`)

Problem. The capability probe is centralized, but the *policy* — which segmenter, in what fallback order — is hand-written twice with different fallback targets: AtlasInput's cascade (native SAM3 → SegFormer → heuristic) vs AtlasSegmentedSDXLInpaint's (native SAM3 → third-party SAM3Segment). The second site's own comment admits it duplicates the first. The two sources also disagree on mask slot index and instance ordering — tribal knowledge currently held in comments.

Solution. A single cascade helper in the adapter layer — input: prompt + desired fallback tier; output: the graph-node reference + which path fired — used by both call sites. The differing fallback tails become explicit named policies, not divergent copies.

Benefits. When SAM3/transformers requirements shift again (they have three times already: triton removal, transformers>=5.5.4, MPS retry), one function changes. Report strings ("which path ran") become consistent for free. ~20 duplicated lines deleted, and the slot-index difference gets one authoritative home.

5 · A mesh-build spec for `build_relief_mesh`'s 24-parameter interface

Worth exploring

core/relief_mesh.py:174-200

7 production call sites (nodes_geometry x3, nodes_inpaint, nodes_export, nodes_depth, relief_mesh internal)

Problem. The implementation is deep, but the interface is wide: 24 parameters, and 7 production call sites each reconstruct the same `view_matrix/fx/fy/cx/cy/scale/horizon_y` bundle from a solve. Every caller carries the full coupling; adding a mesh parameter touches every site (the live-fill drift in candidate 1 is this same disease one layer down).

Benefits / caution. A camera/solve-derived spec object (built once per node via the existing `_solve_camera_params`) would shrink each call to "spec + the 2-3 params this node actually varies". Caution: a config object can become a junk drawer — worth doing only as the camera-bundle half (the part that is always identical), leaving genuinely per-call knobs as keywords. Pairs naturally with candidate 1; probably done *during* it rather than separately.

6 · Small seams: model-id predicates & assess-output indices

Speculative

depth_estimator.py:608,656

nodes_viewport.py:749,808,930,941,974

(a) **Model-family predicates.** `_is_moge_model/_is_da3_model` exist but aren't the sole authority — the cache key, the V2 path, and `nodes_viewport.py:749` re-do ad-hoc `.lower()` substring checks. Cheap fix: import the predicates everywhere. Low urgency, low cost.

(b) **AtlasInput → AtlasAssessImage magic indices.** Wiring by raw output index (`v1m.out(4+i)`, `out(8+i)`, `out(12+i)`) couples AtlasInput to AtlasAssessImage's RETURN_TYPES order — already documented as "if outputs reorder, wiring must follow". A named-slot constant map on

AtlasAssessImage would turn a silent misw wire into a loud KeyError. Speculative because RETURN_TYPES order is already frozen by the saved-workflow contract; the constant is documentation-as-code more than protection.

Top recommendation — Candidate 1, then 2 rides its coattails

Candidate 1 is the highest-leverage move: the drift it prevents **happened this week** (the sawtooth widget missed the proj node; a dead import marked the spot), the area is the hottest in the history, and the fix is fold-onto-existing-helpers rather than new abstraction — the shared trio and `_replace_proxy_role_geometry` already exist and are proven by five sibling nodes. It also settles the cached-vs-uncached ground-scale inconsistency as a side effect.

Candidate 2 is the same disease in core and the bigger line-count win (~140 lines deleted), but it touches the documented "left untouched" decision, so it wants the parity-test-first treatment and a deliberate reopening of that decision — best done as a follow-up once the node-layer fold has demonstrated the pattern.

Generated by /improve-codebase-architecture · evidence from two Explore agents (file:line refs verified against working tree @ main, uncommitted sawtooth work included)